

Covid_19 - Sentiment Analysis

Import Libraries

```
In [15]: import os          #for interacting with the operating system.
import re
import random
import string      # for string operations
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
import plotly.express as px

import warnings
warnings.filterwarnings('ignore')
```

```
In [16]: # SetUp NLTK
```

```
!pip install --user -U nltk
```

Requirement already satisfied: nltk in c:\users\home\appdata\roaming\python\python313\site-packages (3.9.4)
Requirement already satisfied: click in c:\users\home\anaconda3\lib\site-packages (from nltk) (8.2.1)
Requirement already satisfied: joblib in c:\users\home\anaconda3\lib\site-packages (from nltk) (1.5.2)
Requirement already satisfied: regex>=2021.8.3 in c:\users\home\anaconda3\lib\site-packages (from nltk) (2025.9.1)
Requirement already satisfied: tqdm in c:\users\home\anaconda3\lib\site-packages (from nltk) (4.67.1)
Requirement already satisfied: colorama in c:\users\home\anaconda3\lib\site-packages (from click->nltk) (0.4.6)

```
In [17]: import nltk
nltk.download('punkt')
```

```
[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\Home\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!
```

```
Out[17]: True
```

```
In [18]: nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt_tab to
[nltk_data] C:\Users\Home\AppData\Roaming\nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
```

```
Out[18]: True
```

```
In [ ]:
```

Uploading the Dataet

```
In [19]: df = pd.read_csv("Corona_NLP_test.csv")
df.head()
```

```
Out[19]:
```

	UserName	ScreenName	Location	TweetAt	OriginalTweet	Sentiment
0	1	44953	NYC	02-03-2020	TRENDING: New Yorkers encounter empty supermar...	Extremely Negative
1	2	44954	Seattle, WA	02-03-2020	When I couldn't find hand sanitizer at Fred Me...	Positive
2	3	44955	NaN	02-03-2020	Find out how you can protect yourself and love...	Extremely Positive
3	4	44956	Chicagoland	02-03-2020	#Panic buying hits #NewYork City as anxious sh...	Negative
4	5	44957	Melbourne, Victoria	03-03-2020	#toiletpaper #dunnypaper #coronavirus #coronav...	Neutral

```
In [20]: df.shape
```

```
Out[20]: (3798, 6)
```

```
In [ ]:
```

Exploring the labels and Data Visualizatin

```
In [21]: fig = px.pie(df, names='Sentiment',hole=0.3, title='Sentiment Pie Chart')
fig.show()
```

Sentiment Pie Chart



```
In [22]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3798 entries, 0 to 3797
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   UserName        3798 non-null   int64
 1   ScreenName       3798 non-null   int64
 2   Location         2964 non-null   object
 3   TweetAt         3798 non-null   object
 4   OriginalTweet    3798 non-null   object
 5   Sentiment        3798 non-null   object
dtypes: int64(2), object(4)
memory usage: 178.2+ KB
```

```
In [23]: # Check tweets distribution
def pie_chart(df):
    # Converting pd object to list of string
    label_types = df.Sentiment.unique().astype(str)

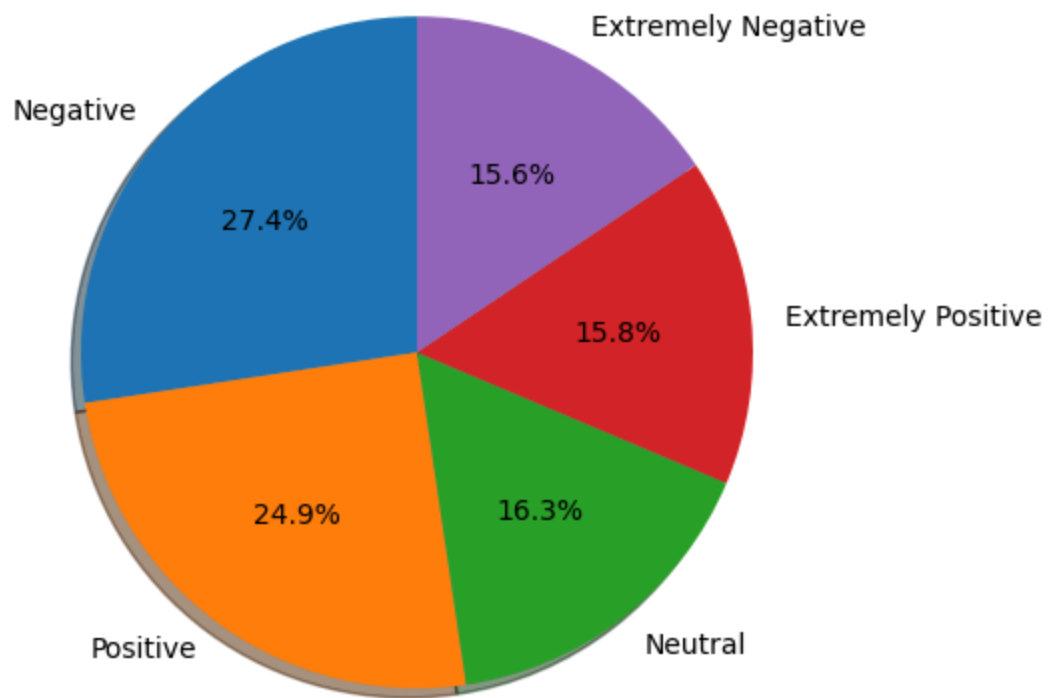
    # Count tweets for each Label
    label_counts = df['Sentiment'].value_counts()
    percentages = (label_counts / label_counts.sum() * 100).round(2)
    print('Labels in the dataset: ', label_types)
    print(pd.DataFrame({
        'Count': label_counts,
        'Percentage': percentages
    })))

    # Pie-chart-----
    plt.pie(
        label_counts.values,
        labels=label_counts.index,
        autopct='%1.1f%%',
        shadow=True,
        startangle=90)

    plt.axis('equal')
    plt.show()
    pie_chart(df)
```

Labels in the dataset: ['Extremely Negative' 'Positive' 'Extremely Positive' 'Negative' 'Neutral']

	Count	Percentage
Sentiment		
Negative	1041	27.41
Positive	947	24.93
Neutral	619	16.30
Extremely Positive	599	15.77
Extremely Negative	592	15.59



In []:

In [24]: `df.head()`

Out[24]:

	UserName	ScreenName	Location	TweetAt	OriginalTweet	Sentiment
0	1	44953	NYC	02-03-2020	TRENDING: New Yorkers encounter empty supermar...	Extremely Negative
1	2	44954	Seattle, WA	02-03-2020	When I couldn't find hand sanitizer at Fred Me...	Positive
2	3	44955	NaN	02-03-2020	Find out how you can protect yourself and love...	Extremely Positive
3	4	44956	Chicagoland	02-03-2020	#Panic buying hits #NewYork City as anxious sh...	Negative
4	5	44957	Melbourne, Victoria	03-03-2020	#toiletpaper #dunnypaper #coronavirus #coronav...	Neutral

In [25]: `df[df['Sentiment'] == 'Negative']['OriginalTweet'].shape`

Out[25]: (1041,)

In []:

Splitting the dataset into training and testing

```
In [26]: # Separating positive_tweets and negative_tweets
all_positive_tweets = list(df[df['Sentiment'] == 'Positive']['OriginalTweet'].astyp
all_negative_tweets = list(df[df['Sentiment'] == 'Negative']['OriginalTweet'].astyp
```

```
In [27]: # train and test split

positive_train, positive_test = train_test_split(all_positive_tweets, test_size=0.2
negative_train, negative_test = train_test_split(all_negative_tweets, test_size=0.2

print("Size of training dataset: ", len(positive_train) + len(negative_train))
print("Size of testing dataset: ", len(positive_test) + len(negative_test))

# print positive in green
print('\033[92m' + '\npositive in green: ')
print('\033[92m' + all_positive_tweets[random.randint(0,947)])
print('\033[91m' + '\nnegative in red: ')
# print negative in red
print('\033[91m' + all_negative_tweets[random.randint(0,947)])
```

Size of training dataset: 1514

Size of testing dataset: 380

positive in green:

Thinking of my friends and family who are working hard to stock our shelves, take care of our sick, deliver our mail (which contains more food, medications, and supplies), and everyone else who selflessly are putting themselves at risk for their fellow man. #coronavirus <https://t.co/G1Q911Nc38>

negative in red:

Now that @GovernorVA has declared a state of emergency VA's price gouging protections are now in effect. If you see any exceptionally high prices for #coronavirus necessities let my office know. Price gouging will not be tolerated in VA

Learn more here?

<https://t.co/3NfOD5zOYY>

In []:

Preprocessing Data

Break down the text into tokens (words).

```
In [33]: from nltk.tokenize import word_tokenize

tweet = all_positive_tweets[12]
words = word_tokenize(tweet)
print('Using NLTK :', words)
```

```
Using NLTK : ['Consumers', 'have', 'increased', 'their', 'online', 'shopping', 'due', 'to', 'coronavirus', '.', 'https', ':', '//t.co/5mYfz3RAD0', '#', 'retail', '#', 'ecommerce', '#', 'study', '#', 'coronavirus', 'https', ':', '//t.co/Dz3H6zrWUT']
```

In [34]: *# tokenize into a Sentence*

```
from nltk.tokenize import sent_tokenize
sentence_splits = sent_tokenize(tweet)
print('Using NLTK :', sentence_splits)
```

```
Using NLTK : ['Consumers have increased their online shopping due to coronavirus.', 'https://t.co/5mYfz3RAD0 #retail #ecommerce #study #coronavirus https://t.co/Dz3H6zrWUT']
```

A lot of unnecessary words like hashtags, hyperlinks are tokenized.

In []:

Reduce Dataset size

```
In [35]: import nltk                                # Python Library for NLP
          from nltk.corpus import twitter_samples    # sample Twitter dataset from NLTK
          from nltk.corpus import stopwords          # module for stop words that come with NLTK
          from nltk.stem import PorterStemmer        # module for stemming
          from nltk.stem import WordNetLemmatizer     # module for Lemmatization

          from nltk.tokenize import TweetTokenizer
          nltk.download('wordnet')
          nltk.download('omw-1.4')
```

```
[nltk_data] Downloading package wordnet to
[nltk_data]   C:\Users\Home\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to
[nltk_data]   C:\Users\Home\AppData\Roaming\nltk_data...
[nltk_data]   Package omw-1.4 is already up-to-date!
```

Out[35]: True

In [37]: *# Removing hyperlinks and hashtags*

```
print('\033[92m' + tweet)
print('\033[94m')

# remove old style retweet text "RT"
tweet2 = re.sub(r'^RT[\s]+', '', tweet)

# remove hyperlinks
tweet2 = re.sub(r'https?:\/\/[^\s\r\n]+', '', tweet2)

# remove hashtags (removing the hash # sign from the word)
tweet2 = re.sub(r'#', '', tweet2)
print(tweet2)
```

Consumers have increased their online shopping due to coronavirus. <https://t.co/5mYfz3RAD0> #retail #ecommerce #study #coronavirus <https://t.co/Dz3H6zrWUT>

Consumers have increased their online shopping due to coronavirus. retail ecommerce study coronavirus

```
In [40]: # Tokenizing

print()
print('\033[92m' + tweet2)
print('\033[94m')

# instantiate tokenizer class
tokenizer = TweetTokenizer(preserve_case=False, strip_handles=True, reduce_len=True)

# tokenize tweets
tweet_tokens = tokenizer.tokenize(tweet2)

print()
print('Tokenized string:')
print(tweet_tokens)
```

Consumers have increased their online shopping due to coronavirus. retail ecommerce study coronavirus

Tokenized string:

['consumers', 'have', 'increased', 'their', 'online', 'shopping', 'due', 'to', 'coronavirus', '.', 'retail', 'ecommerce', 'study', 'coronavirus']

```
In [41]: # Import the english stop words list from NLTK

nltk.download("stopwords")
stopwords_english = stopwords.words('english')

print('Stop words\n')
print(stopwords_english)

print('\nPunctuation\n')
print(string.punctuation)
```

Stop words

```
[ 'a', 'about', 'above', 'after', 'again', 'against', 'ain', 'all', 'am', 'an', 'and', 'any', 'are', 'aren', "aren't", 'as', 'at', 'be', 'because', 'been', 'before', 'being', 'below', 'between', 'both', 'but', 'by', 'can', 'couldn', "couldn't", 'd', 'did', 'didn', "didn't", 'do', 'does', 'doesn', "doesn't", 'doing', 'don', "don't", 'down', 'during', 'each', 'few', 'for', 'from', 'further', 'had', 'hadn', "hadn't", 'has', 'hasn', "hasn't", 'have', 'haven', "haven't", 'having', 'he', "he'd", "he'll", 'her', 'here', 'hers', 'herself', "he's", 'him', 'himself', 'his', 'how', 'i', "i'd", 'if', "i'll", "i'm", 'in', 'into', 'is', 'isn', "isn't", 'it', "it'd", "it'll", "it's", 'its', 'itself', "i've", 'just', 'll', 'm', 'ma', 'me', 'mightn', "mightn't", 'more', 'most', 'mustn', "mustn't", 'my', 'myself', 'needn', "needn't", 'no', 'nor', 'not', 'now', 'o', 'of', 'off', 'on', 'once', 'only', 'or', 'other', 'our', 'ours', 'ourselves', 'out', 'over', 'own', 're', 's', 'same', 'shan', "shan't", 'she', "she'd", "she'll", "she's", 'should', 'shouldn', "shouldn't", "should've", 'so', 'some', 'such', 't', 'than', 'that', "that'll", 'the', 'their', 'theirs', 'them', 'themselves', 'then', 'there', 'these', 'they', "they'd", "they'll", "they're", "they've", 'this', 'those', 'through', 'to', 'too', 'under', 'until', 'up', 've', 'very', 'was', 'wasn', "wasn't", 'we', "we'd", "we'll", "we're", 'were', 'weren', "weren't", "we've", 'what', 'when', 'where', 'which', 'while', 'who', 'whom', 'why', 'will', 'with', 'won', "won't", 'wouldn', "wouldn't", 'y', 'you', "you'd", "you'll", 'your', "you're", 'yours', 'yourself', 'yourselves', "you've"]
```

Punctuation

```
!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~
```

```
[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\Home\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!
```

```
In [42]: print()
print('\033[92m')
print(tweet_tokens)
print('\033[94m')

tweets_clean = []
#IP=["i", "am", "great"]
for word in tweet_tokens: # Go through every word in your tokens list
    if (word not in stopwords_english and # remove stopwords
        word not in string.punctuation): # remove punctuation
        tweets_clean.append(word)
#OP=["great"]
print('removed stop words and punctuation:')
print(tweets_clean)
```

```
['consumers', 'have', 'increased', 'their', 'online', 'shopping', 'due', 'to', 'coronavirus', '.', 'retail', 'ecommerce', 'study', 'coronavirus']
```

```
removed stop words and punctuation:
```

```
['consumers', 'increased', 'online', 'shopping', 'due', 'coronavirus', 'retail', 'ecommerce', 'study', 'coronavirus']
```

```
In [ ]:
```


Stemming & Lemmatization

```
In [ ]: # Stemming

print()
print('\033[92m')
print(tweets_clean)
print('\033[94m')

# Instantiate stemming class
stemmer = PorterStemmer()

# Create an empty list to store the stems
tweets_stem = []

for word in tweets_clean:
    stem_word = stemmer.stem(word) # stemming word
    tweets_stem.append(stem_word) # append to the list

print('stemmed words:')
print(tweets_stem)
```

```
In [43]: ## Lemmatization

def process_tweet(tweet):
    #Clean #Remove stopwords/punc #Normalize using Lemmatizer
    lemmatizer = WordNetLemmatizer()
    stopwords_english = stopwords.words('english')

    tweet = re.sub(r'\$\w*', '', tweet) #1 remove stock market tickers like $GE
    tweet = re.sub(r'^RT[\s]+', '', tweet) #2 remove old style retweet text "RT"
    tweet = re.sub(r'https?:\/\/[^\s\r\n]+', '', tweet) #3 remove hyperlinks
    tweet = re.sub(r'#', '', tweet) #4 remove hashtags (removing the hash # sign f
    tokenizer = TweetTokenizer(preserve_case=False, strip_handles=True, reduce_len=T
    tweet_tokens = tokenizer.tokenize(tweet)

    tweets_clean = []
    for word in tweet_tokens: #6 Stopwords and Punctuation Remover
        if (word not in stopwords_english and # remove stopwords
            word not in string.punctuation): # remove punctuation
            lemma_word = lemmatizer.lemmatize(word) # 7 Normalization - # stemmin
            tweets_clean.append(lemma_word)

    return tweets_clean
```

```
In [54]: # choose the same tweet

tweet = all_positive_tweets[12]

print()
print('\033[92m')
print(tweet)
print('\033[94m')
```

```
# call the imported function
tweets_stem = process_tweet(tweet); # Preprocess a given tweet

print('preprocessed tweet:')
print(tweets_stem) # Print the result
```

Consumers have increased their online shopping due to coronavirus. <https://t.co/5mYfz3RAD0> #retail #ecommerce #study #coronavirus <https://t.co/Dz3H6zrWUT>

```
preprocessed tweet:
['consumer', 'increased', 'online', 'shopping', 'due', 'coronavirus', 'retail', 'ecommerce', 'study', 'coronavirus']
```

In []:

Preprocessed tweet to train a model

Build Frequency Dictionary

```
In [55]: def build_freqs(tweets, ys):
        """
        tweets: a list of tweets
        ys: an m x 1 array with the sentiment label of each tweet
            (either 0 or 1)
        freqs: a dictionary mapping each (word, sentiment) pair (=key) to its frequency
        """
        # Convert np array to list since zip needs an iterable.
        # The squeeze is necessary or the list ends up with one element.
        # Also note that this is just a NOP if ys is already a list.
        yslist = np.squeeze(ys).tolist()
        #Y
        #[0,1,0,1,1,0] -> Sentiments ->Mx1

        # Start with an empty dictionary and populate it by looping over all tweets
        # and over all processed words in each tweet.
        freqs = {}
        for y, tweet in zip(yslist, tweets):
            for word in process_tweet(tweet):
                pair = (word, y) #(happy,1)->3, (happy,0)->0
                if pair in freqs:
                    freqs[pair] += 1
                else:
                    freqs[pair] = 1

        return freqs
```

```
In [56]: # make a numpy array representing labels of the tweets

training_tweets = positive_train + negative_train
labels = np.append(np.ones((len(positive_train))), np.zeros((len(negative_train))))
```

```
In [57]: # create frequency dictionary
freqs = build_freqs(training_tweets, labels)
```

```
# check data type
print(f'type(freqs) = {type(freqs)}')

# check length of the dictionary
print(f'len(freqs) = {len(freqs)}')
```

```
type(freqs) = <class 'dict'>
len(freqs) = 7964
```

In []:

Extracting features function

```
In [49]: def extract_features(tweet, freqs):
    '''
        tweet: a list of words for one tweet
        freqs: a dictionary corresponding to the frequencies of each tuple (word, 1
        x: a feature vector of dimension (1,3)
    '''
    # process_tweet tokenizes, stems, and removes stopwords
    word_l = process_tweet(tweet)
    # 3 elements in the form of a 1 x 3 vector
    x = np.zeros((1, 3))

    # bias term is set to 1
    x[0,0] = 1

    # loop through each word in the list of words
    for word in word_l:

        # increment the word count for the positive label 1
        x[0,1] += freqs.get((word, 1.0),0)

        # increment the word count for the negative label 0
        x[0,2] += freqs.get((word, 0.0),0)

    assert(x.shape == (1, 3))
    return x
```

```
In [58]: # feature extraction for a tweet

train_x = training_tweets
tmp1 = extract_features(train_x[0], freqs)
print(tmp1)
```

```
[[1.00e+00 2.26e+03 2.44e+03]]
```

```
In [59]: #feature extraction for a random string

tmp2 = extract_features('This batch is the best batch', freqs)
print(tmp2)
```

```
[[ 1. 11.  3.]]
```

```
In [60]: # collect the features 'x' and stack them into a matrix 'X'
X = np.zeros((len(train_x), 3))
for i in range(len(train_x)):
    X[i, :] = extract_features(train_x[i], freqs)

# training labels corresponding to X
train_y = np.append(np.ones((len(positive_train), 1)), np.zeros((len(negative_train), 1)))
Y = np.ravel(train_y, order='C')
```

```
In [ ]:
```

LogisticRegression to classify

```
In [61]: from sklearn.linear_model import LogisticRegression
clf = LogisticRegression().fit(X, Y)

print("Training Accuracy: ", clf.score(X, Y))
```

Training Accuracy: 0.6723910171730515

```
In [62]: # Logistic Regression with L1 Regularization

clf = LogisticRegression(C = 0.1, solver= 'saga', penalty= 'l1', max_iter = 500).fit(X, Y)

print("Training Accuracy: ", clf.score(X, Y))
```

Training Accuracy: 0.6829590488771466

Testing

```
In [63]: testing_tweets = positive_test + negative_test

test_X = np.zeros((len(testing_tweets), 3))
for i in range(len(testing_tweets)):
    test_X[i, :] = extract_features(testing_tweets[i], freqs)

# training labels corresponding to X
test_y = np.append(np.ones((len(positive_test), 1)), np.zeros((len(negative_test), 1)))
test_Y = np.ravel(test_y, order='C')

print("Testing Accuracy: ", clf.score(test_X, test_Y))
```

Testing Accuracy: 0.5789473684210527

```
In [ ]:
```

Predict whether a tweet is positive or negative.

Classify 1 for a positive sentiment, and 0 for a negative sentiment.

```
In [64]: def predict_custom_tweet(tweet, freqs):
x = extract_features(tweet, freqs)
```

```

y_pred = clf.predict(x)
return y_pred

# test your function
list_of_tweets = ['\033[92m'+ 'Thank God coronavirus is over',
                  '\033[91m'+ 'This is seriously ridiculous. Stop hoarding',
                  '\033[92m'+ 'Government efforts are great.',
                  '\033[93m'+ 'Nonsense', #anomaly in classification
                  '\033[91m'+ 'Panic-buying is pushing up prices.#coronavirus',
                  '\033[92m'+ 'THANK YOU to make hand sanitizer in distillery amid o

for tweet in list_of_tweets:
    print( '%s -> %f' % (tweet, predict_custom_tweet(tweet, freqs)))

```

Thank God coronavirus is over -> 1.000000

This is seriously ridiculous. Stop hoarding -> 0.000000

Government efforts are great. -> 1.000000

Nonsense -> 0.000000

Panic-buying is pushing up prices.#coronavirus -> 0.000000

THANK YOU to make hand sanitizer in distillery amid outbreak -> 1.000000

Conclusion: Test Accuracy is 57.89 which means the model correctly classifies about 58 out of every 100 tweets. This is only slightly better than random guessing for a balanced binary classification problem. However, there is room for improvement through better preprocessing, feature engineering, or a stronger model.

In []:

In []:

In []:

In []:

In []:

In []: